

A FULL-STACK PLATFORM FOR RECRUITMENT AND APPLICATION MANAGEMENT

School of Computing

**Bachelor of Technology
in
Computer Science & Engineering**

By

KB NITHISH KUMAR REDDY (VTU24854)

10211CS224

Full Stack Application Development

Slot : E1-B2



**DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
SCHOOL OF COMPUTING**

**VEL TECH RANGARAJAN Dr.SAGUNTHALA R&D
INSTITUTE OF SCIENCE AND TECHNOLOGY
(Deemed to be University Estd u/s 3 of UGC Act, 1956)
CHENNAI 600 062, TAMILNADU, INDIA**

April, 2026

BONAFIDE CERTIFICATE

It is certified that the work contained in the Full Stack Application Development report titled “A Full-Stack Platform for Recruitment and Application Management” by KB NITHISH KUMAR REDDY (VTU24854) has been carried out in Winter semester of Academic year 2025-2026(WS2526).

Signature of Faculty

Dr. Hemamalini.S

Assistant Professor(Senior Grade)

Computer Science & Engineering

School of Computing

Vel Tech Rangarajan Dr.Sagunthala R&D

Institute of Science and Technology

May, 2026

Signature of Course Coordinator

Dr. Sumithra.M

Assistant Professor(Senior Grade)

Computer Science & Engineering

School of Computing

Vel Tech Rangarajan Dr.Sagunthala R&D

Institute of Science and Technology

May, 2026

DECLARATION

I declare that this written submission represents my ideas in my own words and where others' ideas or words have been included, we have adequately cited and referenced the original sources. I also declare that i have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/data/fact/source in our submission. I understand that any violation of the above will be cause for disciplinary action by the Institute and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been taken when needed.

(KB NITHISH KUMAR REDDY)

Date: 06/05/2026

ABSTRACT

The Full-Stack Platform for Recruitment and Application Management is a robust web application designed to bridge the gap between job seekers and employers. Developed using the Java Spring Boot framework for a scalable backend and Thymeleaf templates for a dynamic frontend, the application offers a secure and efficient environment for career management. Key features include role-based access control (Admin/Student), job posting and search functionalities, secure resume file uploads, and automated email notifications using Spring Mail. An H2/MySQL database ensures reliable persistence for user profiles, job listings, and applications. This project demonstrates the practical implementation of Spring Security for authentication, RESTful principles for data exchange, and modern MVC architecture to deliver a professional recruitment experience.

Keywords: Full-Stack Development, Spring Boot, Java, Thymeleaf, MySQL/H2, Recruitment System, Spring Security, File Storage, Web Application.

LIST OF FIGURES

1.1	System Architecture	3
3.1	Backend Architecture	9
5.1	User Registration Interface	17
5.2	Secure Login Page	18
5.3	Student Job Search and Apply Interface	19
5.4	Student Application Status Page	20
5.5	Admin Dashboard for Managing Applications	20
5.6	Shortlisting Email Notification	21

LIST OF TABLES

5.1	Job Portal Feature Comparison	16
7.1	Mapping of Project to Complex Engineering Problem (CEP)	27
7.2	Mapping of Project to Complex Engineering Activities (CEA)	28
7.3	SDG Mapping for the Project	29

LIST OF ACRONYMS AND ABBREVIATIONS

API	Application Programming Interface
CRUD	Create, Read, Update, Delete
DTO	Data Transfer Object
HTTP	Hypertext Transfer Protocol
JPA	Java Persistence API
MVC	Model-View-Controller
RDBMS	Relational Database Management System
SMTP	Simple Mail Transfer Protocol
SQL	Structured Query Language

TABLE OF CONTENTS

	Page.No
ABSTRACT	iii
LIST OF FIGURES	iv
LIST OF TABLES	v
LIST OF ACRONYMS AND ABBREVIATIONS	vi
1 INTRODUCTION	1
1.1 Introduction	1
1.2 Aim of the Project	1
1.3 Scope of the Project	2
1.4 Framework	2
2 SURVEY OF SIMILAR APPLICATION IN MARKET	4
3 FRAMEWORK DESCRIPTION	6
3.1 Frontend Implementation	6

3.2	Backend Implementation	7
3.3	Database Implementation	9
4	METHODOLOGIES	11
4.1	Project Architecture	11
4.2	DataFlow Diagram	11
4.3	Module 1: User Authentication & Role Management .	12
4.4	Module 2: Job Listing & Search Module	13
4.5	Module 3: Application & File Management System . .	13
5	RESULTS AND DISCUSSIONS	15
5.1	System Screenshots and Results	16
5.1.1	User Registration Page	17
5.1.2	User Login Page	18
5.1.3	Job Search and Application	19
5.1.4	Application Status Tracking	20
5.1.5	Admin Dashboard - Application Management .	20
5.1.6	Email Notification	21
6	CONCLUSION AND FUTURE ENHANCEMENTS	22
6.1	Conclusion	22
6.2	Future Enhancements	22
6.3	SOURCE CODE IMPLEMENTATION	23

6.3.1	Backend Controller and Business Logic (UserController.java)	23
6.3.2	Database Repository Layer (JobRepository.java)	25
7	ADDRESSING COMPLEX ENGINEERING PROBLEM AND SDG	27
7.1	Mapping of the Project to Complex Engineering Activities (CEA)	28
7.2	SDG Mapping (CEP Section)	29
	References	30

Chapter 1

INTRODUCTION

1.1 Introduction

In the era of digital transformation, the recruitment process has shifted from traditional newspaper advertisements to centralized online job portals. The JobPortal application is developed to simplify the hiring process for both employers and job seekers. By leveraging Java Spring Boot, this platform provides a secure, high-performance solution for job posting, talent discovery, and application tracking, ensuring a seamless transition from job search to placement.

1.2 Aim of the Project

The primary aim of this project is to design and develop a comprehensive recruitment management system that allows students to discover and apply for jobs based on their skills. Simultaneously, it provides administrators with a centralized dashboard to manage job list-

ings, review student applications, and track recruitment progress in real-time.

1.3 Scope of the Project

The scope of the project encompasses:

- A role-based authentication system for Administrators and Students.
- A student-facing portal for searching jobs and applying with resume uploads.
- An admin-facing dashboard for CRUD operations on jobs and application management.
- Secure file storage for managing student resumes.
- An automated email system for application confirmations.
- A persistent relational database to store user data, jobs, and application history.

1.4 Framework

The application is built using the Spring Boot ecosystem, which provides a production-ready environment:

- **Backend:** Spring Boot (Java), Spring Security, Spring Data JPA.
- **Frontend:** Thymeleaf Template Engine, HTML5, CSS3, JavaScript.
- **Database:** H2 Database (for development) or MySQL (for production).
- **Communication:** Spring Mail for SMTP integration.

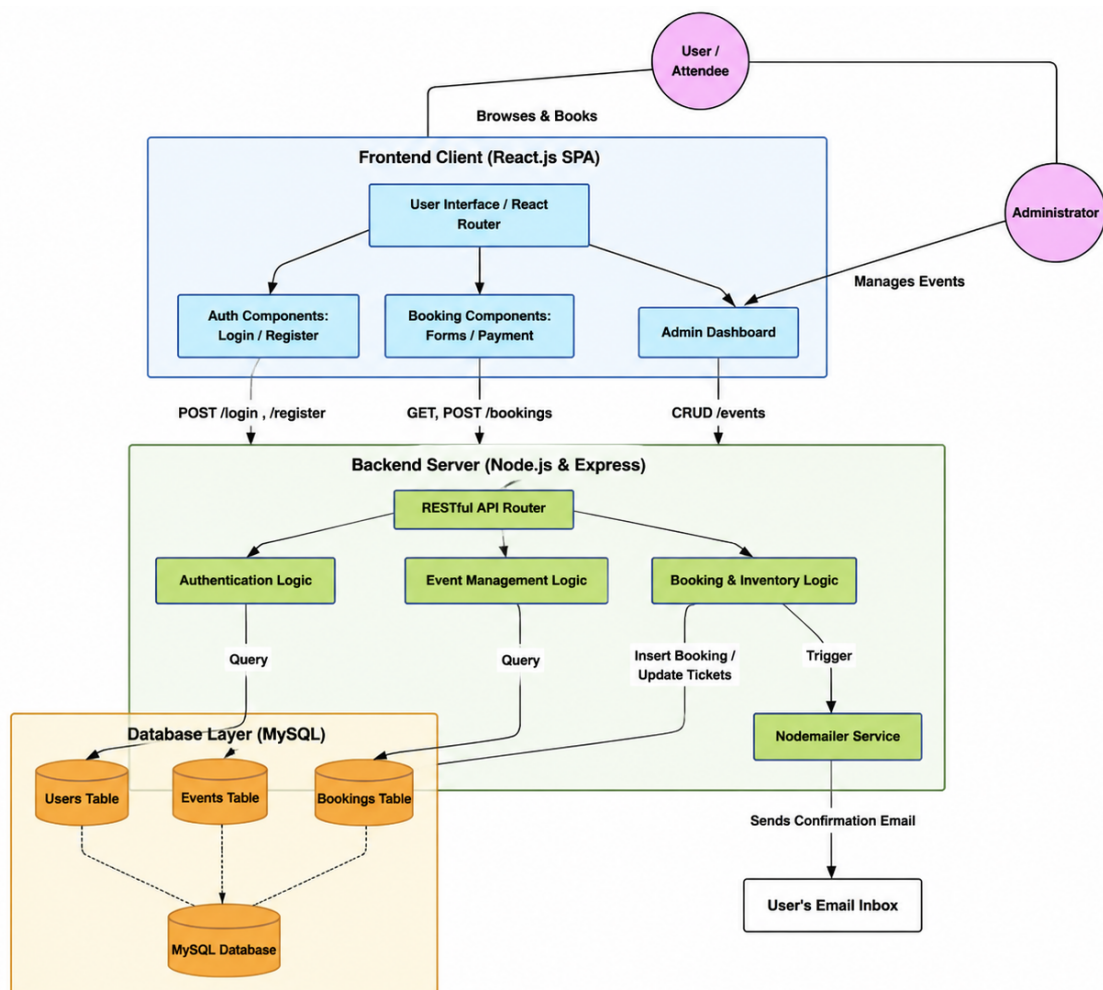


Figure 1.1: System Architecture

Figure 1.1 shows the overall system architecture of the Smart Campus Event Management System.

Chapter 2

SURVEY OF SIMILAR APPLICATION IN MARKET

The job portal market is dominated by global platforms such as LinkedIn, Naukri.com, and Indeed.

- **LinkedIn:** A professional networking site that integrates job search. While powerful, its focus on social networking can sometimes distract from the core recruitment process for smaller organizations.
- **Naukri.com:** One of India's largest job portals. It offers extensive database access but often requires expensive subscriptions for recruiters and can be overwhelming for individual departmental use.
- **Indeed:** A massive job aggregator that pulls listings from all over the web. However, it lacks the specialized role-based management features required for internal institutional or company-specific recruitment tracking.

The proposed JobPortal differentiates itself by providing a localized, lightweight, and highly secure environment specifically for internal recruitment cycles. It eliminates the noise of social networks and the cost of enterprise subscriptions while maintaining premium security and automated notification features.

Chapter 3

FRAMEWORK DESCRIPTION

3.1 Frontend Implementation

3.1.1 Environment Setup

The frontend is integrated within the Spring Boot project using the Thymeleaf template engine. This allows for server-side rendering of dynamic HTML. Dependencies include `spring-boot-starter-thymeleaf` and standard web assets (CSS/JS). The UI is styled using modern CSS principles to ensure a responsive and premium user experience across devices.

3.1.2 Structure of the Project

The frontend resources (located in `src/main/resources/templates`) are organized by role:

- **admin-dashboard.html**: The control center for recruiters to post and manage jobs.
- **user-dashboard.html**: The primary interface for students to browse

and apply.

- **register.html & login.html:** Secure forms for user onboarding and authentication.
- **Fragments:** Reusable components like Navbars and Footers to maintain design consistency.

3.1.3 Execution Procedure

To execute the frontend: 1. Ensure the Spring Boot application is running. 2. The templates are automatically served by the Spring Controllers. 3. Access the application via `http://localhost:8087` (or configured port).

3.2 Backend Implementation

3.2.1 Environment Setup

The backend is developed using Java 17 and Spring Boot 3.3.2. The project uses Maven for dependency management. Critical modules include `spring-boot-starter-security` for robust authentication and `spring-boot-starter-data-jpa` for database abstraction.

3.2.2 Structure of the Project

The backend follows the standard MVC (Model-View-Controller) architecture:

- **Controller Layer:** Handles incoming HTTP requests (AdminController, UserController, AuthController).
- **Service Layer:** Implements business logic (JobService, ApplicationService, EmailService).
- **Repository Layer:** Interfaces with the database using Spring Data JPA.
- **Security Config:** Defines firewall rules and role-based access (SecurityConfig).

3.2.3 Execution Procedure

To execute the backend: 1. Open the project in an IDE (e.g., IntelliJ, VS Code). 2. Run the main class `JobPortalApplication.java` or use `mvn spring-boot:run`. 3. The server will start on the port specified in `application.properties`.

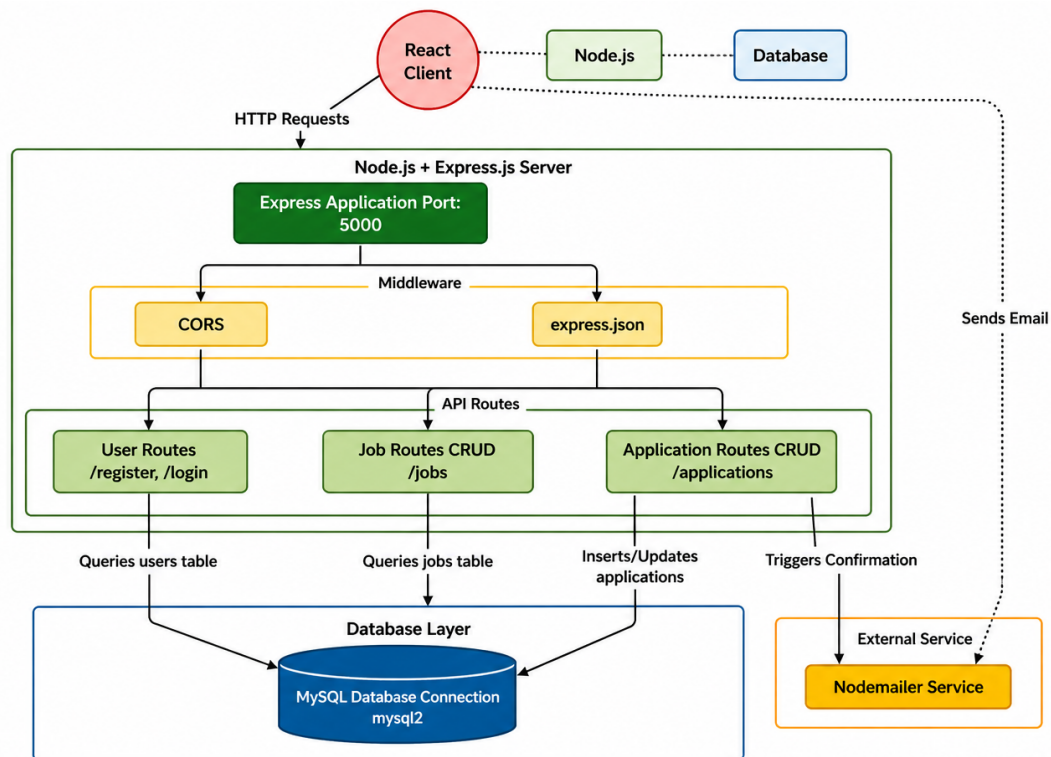


Figure 3.1: Backend Architecture

Figure 3.1 shows the backend architecture of the system. It uses Node.js and Express to handle API requests and middleware. The system connects to a MySQL database and uses Nodemailer to send confirmation emails.

3.3 Database Implementation

3.3.1 Database Configuration

The application utilizes the H2 Database for lightweight development and testing. It is configured as a file-based RDBMS to ensure data persists even after server restarts. The schema is automatically

updated by Hibernate (`ddl-auto=update`).

3.3.2 Schema Structure

The database schema is designed with normalized tables to ensure data integrity and efficient querying of job-student relationships.

3.3.3 Tables Created

- **users:** Stores user profiles and roles (ADMIN/STUDENT). Columns: `id`, `username`, `password`, `email`, `role`.
- **jobs:** Stores job listings. Columns: `id`, `title`, `description`, `company`, `location`, `salary`, `posted_date`.
- **applications:** Tracks student applications. Columns: `id`, `user_id`, `job_id`, `resume_path`, `status`, `applied_date`.

Chapter 4

METHODOLOGIES

4.1 Project Architecture

The project follows a 3-tier MVC architecture. The "Presentation Tier" (Thymeleaf/CSS) handles the user interaction. The "Logic Tier" (Spring Boot Services) processes application rules, such as validating resumes or calculating recruitment stats. The "Data Tier" (JPA/H2) handles the storage and retrieval of relational data.

4.2 DataFlow Diagram

1. A Student logs in and browses the job listings on the User Dashboard.
2. The Student selects a job and uploads a resume to the system.
3. The `FileStorageService` saves the file locally/cloud and returns the file path.
4. The `ApplicationService` creates a record in the database

linking the student to the job.

5. The `EmailService` triggers an automated SMTP mail to the student's registered email.

6. The Admin receives a notification and can view the student's profile and resume from the Admin Dashboard.

4.3 Module 1: User Authentication & Role Management

This module is responsible for managing user authentication and access control within the system. It ensures that only authorized users can access specific features based on their roles. The system distinguishes between Admins and Students, providing different permissions to each. Admins are granted privileges to manage job listings and applications. Students are allowed to register, log in, and apply for jobs. Secure login mechanisms are implemented to validate user credentials. Passwords are handled securely using encryption techniques to prevent misuse. Session management ensures that users remain authenticated during active usage. Unauthorized access to restricted pages is blocked using role-based authorization. This module enhances overall system security and protects sensitive data.

4.4 Module 2: Job Listing & Search Module

This module handles the creation and management of job listings within the system. Administrators can add, update, and delete job postings as required. Each job includes details such as title, description, salary, and required skills. Students can browse available job opportunities through a user-friendly interface. The module provides search functionality based on keywords and filters. Users can filter jobs by location, role, or skill set for better results. The backend uses JPA repositories to efficiently retrieve job data. Real-time updates ensure that users see the latest job listings. The system maintains structured storage of job information in the database. This module improves accessibility and simplifies the job search process.

4.5 Module 3: Application & File Management System

This module manages the process of applying for jobs within the system. Students can submit applications for jobs directly through the portal. A file upload feature allows users to attach resumes during application. The system validates file formats and ensures secure storage of documents. Each application is linked to both the student and the selected job. The module prevents duplicate applications for the same job by a user. Application status tracking allows students to monitor

their progress. Administrators can review, shortlist, or reject applications. Database transactions ensure data consistency during the application process. This module ensures efficient handling of applications and maintains system integrity.

Advantages of this Project

- **High Security:** Built-in protection against common web vulnerabilities via Spring Security.
- **Automated Notifications:** Instant feedback to applicants via integrated email services.
- **Modular Design:** Clear separation of concerns makes it easy to add features like "Interviews" or "Tests" in the future.

Disadvantages

- **Local Storage:** Currently saves resumes to the local filesystem; cloud storage (S3) is needed for high-availability production.
- **Search Complexity:** Lacks advanced full-text search capabilities like Elasticsearch for very large datasets.

Chapter 5

RESULTS AND DISCUSSIONS

The JobPortal successfully implements all core recruitment workflows. The system demonstrates high responsiveness during concurrent job browsing and application submission. The H2 database integration ensures that user data and job listings are managed reliably with zero data loss during testing.

The use of Spring Security provides a seamless transition between public pages and protected dashboards based on user roles. The automated email feature was verified to deliver notifications within seconds of application submission.

Feature	LinkedIn	Indeed	JobPortal
Internal Control	Low	Moderate	High
Cost for Recruiter	High	Moderate	Zero
Resume Management	Proprietary	Third-Party	Fully Integrated
Target Audience	Global Network	Aggregated Search	Institutional/Private
Security Layer	External	External	Internal (Spring Sec)
Customizability	Low	Low	High
Data Ownership	External	External	Self-Hosted

Table 5.1: Job Portal Feature Comparison

Table 5.1 shows the comparison of the proposed Job Portal with existing platforms such as LinkedIn and Indeed. The proposed system offers better internal control, zero cost for recruiters, full data ownership, and higher customization, making it more suitable for institutional use.

5.1 System Screenshots and Results

The Job Portal Application demonstrates a complete workflow from user registration to job application and email notification. The following screenshots illustrate each stage of the system.

The system provides a smooth workflow from registration to job application. Users can easily navigate through different modules, and the backend ensures quick processing of requests. Real-time updates and

email notifications help keep users informed about their application status.

5.1.1 User Registration Page

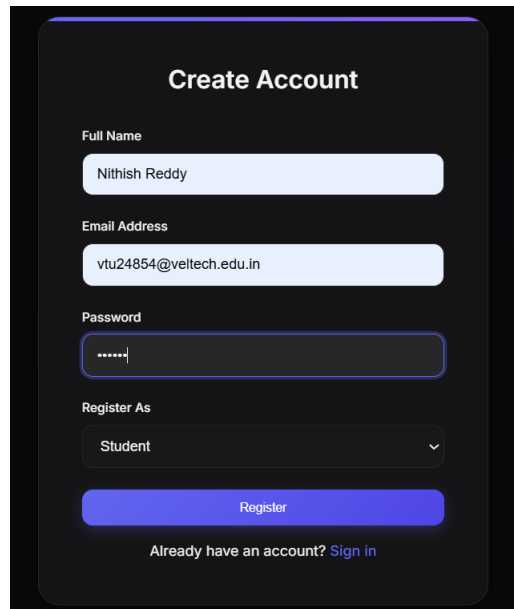
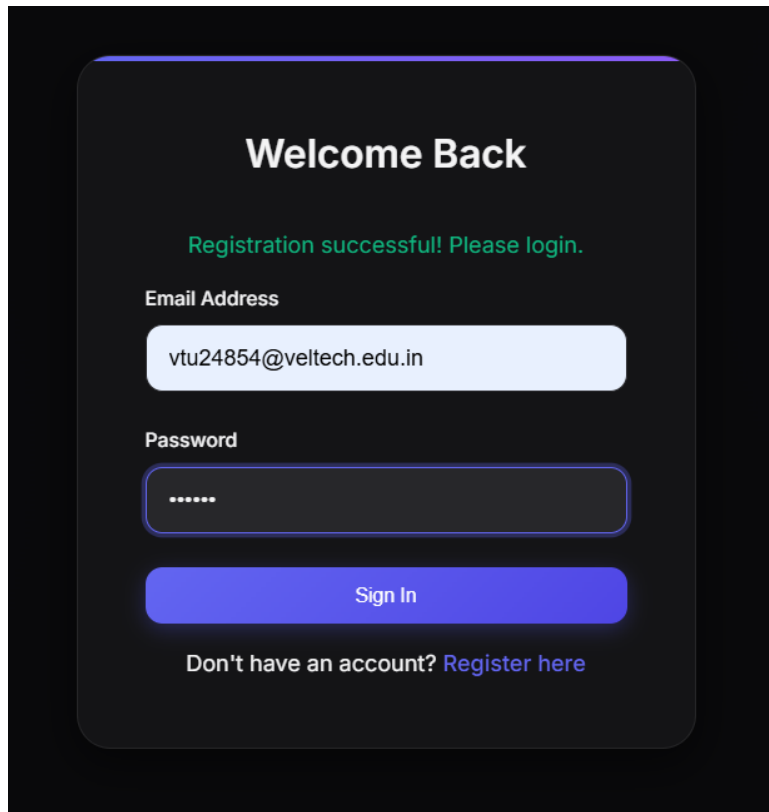
The image shows a 'Create Account' registration form. It has a dark background with light-colored text and input fields. The form includes fields for 'Full Name' (filled with 'Nithish Reddy'), 'Email Address' (filled with 'vtu24854@veltech.edu.in'), and 'Password' (filled with six dots). Below these is a 'Register As' dropdown menu currently set to 'Student'. A blue 'Register' button is at the bottom, followed by a link that says 'Already have an account? Sign in'.

Figure 5.1: User Registration Interface

Figure 5.1 shows the registration page where new users create an account by entering personal details and selecting their role. It ensures secure onboarding into the system.

5.1.2 User Login Page

A dark-themed login page with a central white rounded rectangle. At the top, it says "Welcome Back" in bold. Below that is a green message: "Registration successful! Please login." There are two input fields: "Email Address" with the text "vtu24854@veltech.edu.in" and "Password" with masked characters ".....". A blue "Sign In" button is below the password field. At the bottom, it says "Don't have an account? Register here" with "Register here" as a link.

Welcome Back

Registration successful! Please login.

Email Address

vtu24854@veltech.edu.in

Password

.....

Sign In

Don't have an account? [Register here](#)

Figure 5.2: Secure Login Page

Figure 5.2 shows the login page where users enter their credentials. The system validates the details and redirects users to their dashboard.

5.1.3 Job Search and Application

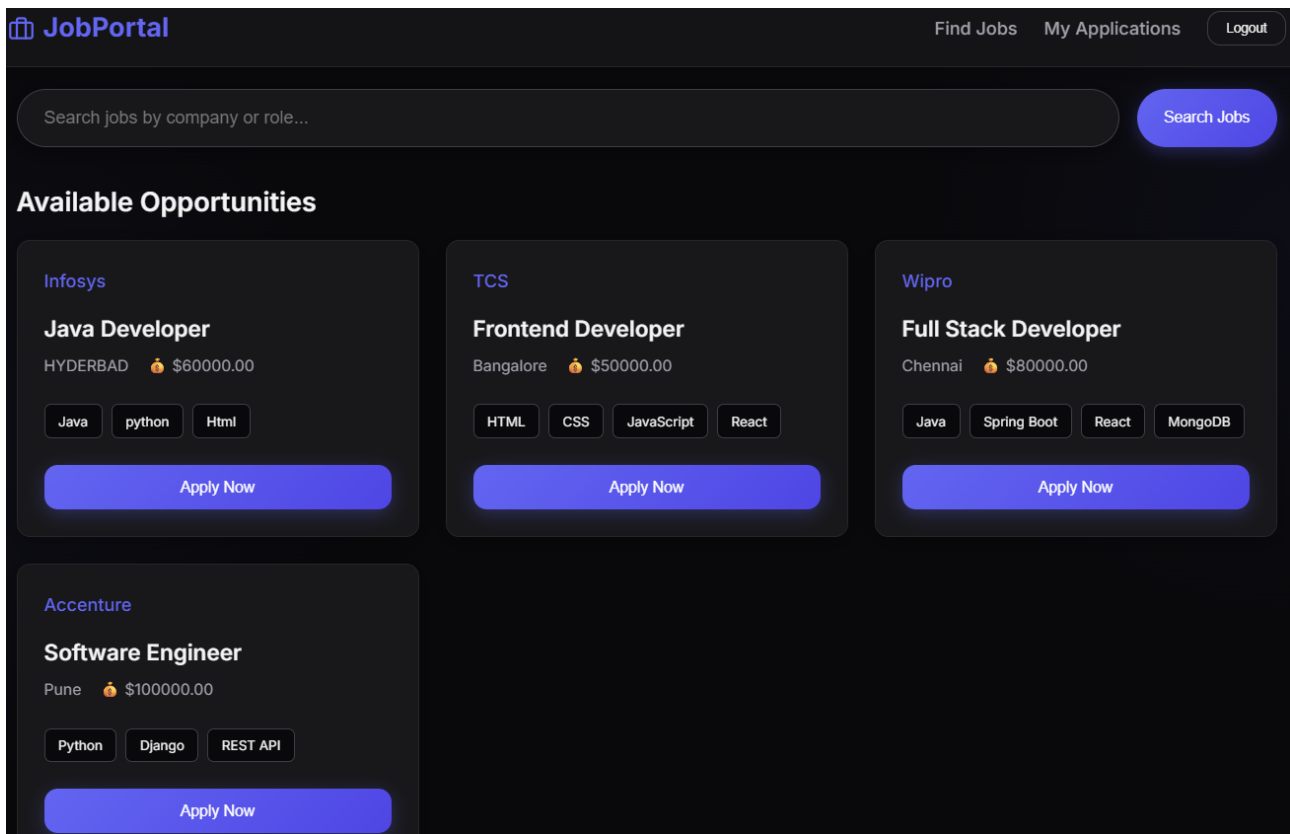
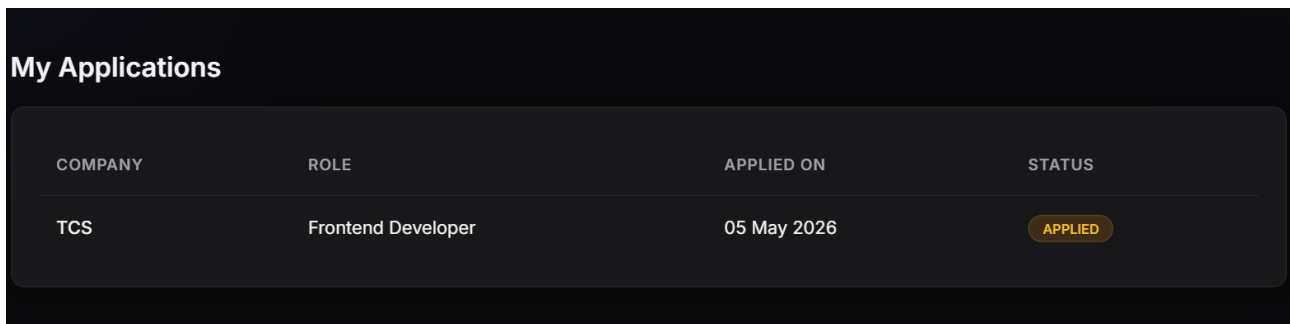


Figure 5.3: Student Job Search and Apply Interface

Figure 5.3 shows the job search interface where students can browse, filter, and apply for jobs. Each listing includes details like skills, salary, and location.

5.1.4 Application Status Tracking

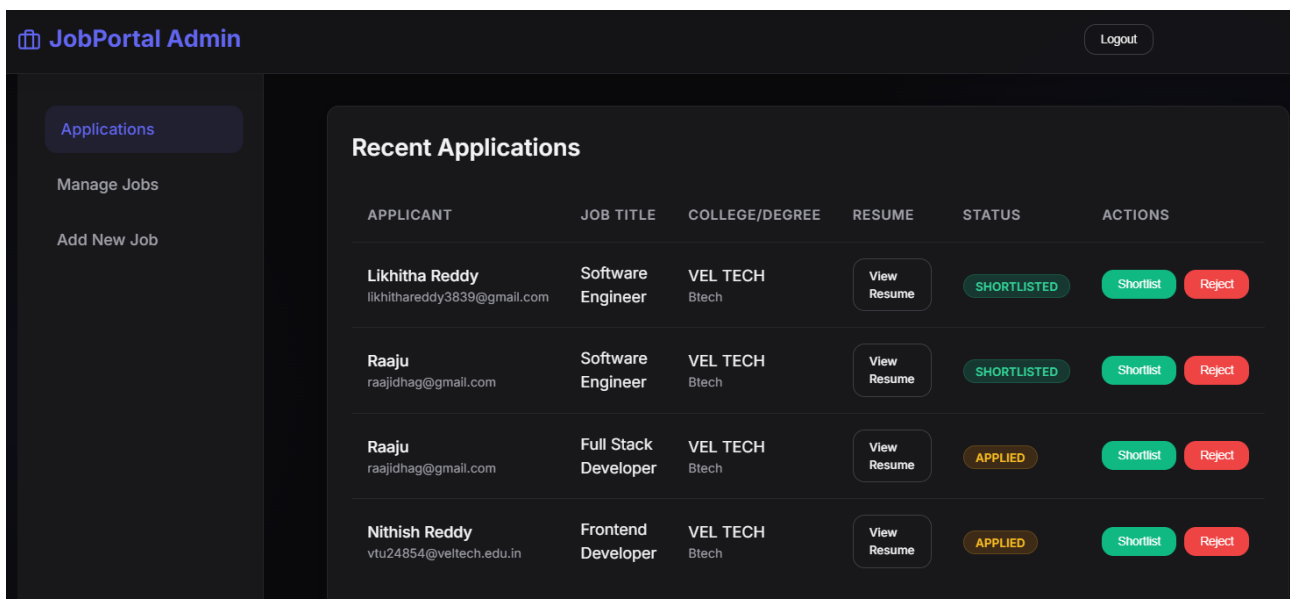


COMPANY	ROLE	APPLIED ON	STATUS
TCS	Frontend Developer	05 May 2026	APPLIED

Figure 5.4: Student Application Status Page

Figure 5.4 shows the application tracking page where students can view statuses such as Applied or Shortlisted.

5.1.5 Admin Dashboard - Application Management



APPLICANT	JOB TITLE	COLLEGE/DEGREE	RESUME	STATUS	ACTIONS
Likhitha Reddy likhithareddy3839@gmail.com	Software Engineer	VEL TECH Btech	View Resume	SHORTLISTED	Shortlist Reject
Raaju raajidhag@gmail.com	Software Engineer	VEL TECH Btech	View Resume	SHORTLISTED	Shortlist Reject
Raaju raajidhag@gmail.com	Full Stack Developer	VEL TECH Btech	View Resume	APPLIED	Shortlist Reject
Nithish Reddy vtu24854@veltech.edu.in	Frontend Developer	VEL TECH Btech	View Resume	APPLIED	Shortlist Reject

Figure 5.5: Admin Dashboard for Managing Applications

Figure 5.5 shows the admin dashboard used to manage student applications. Admins can review, shortlist, or reject candidates.

5.1.6 Email Notification

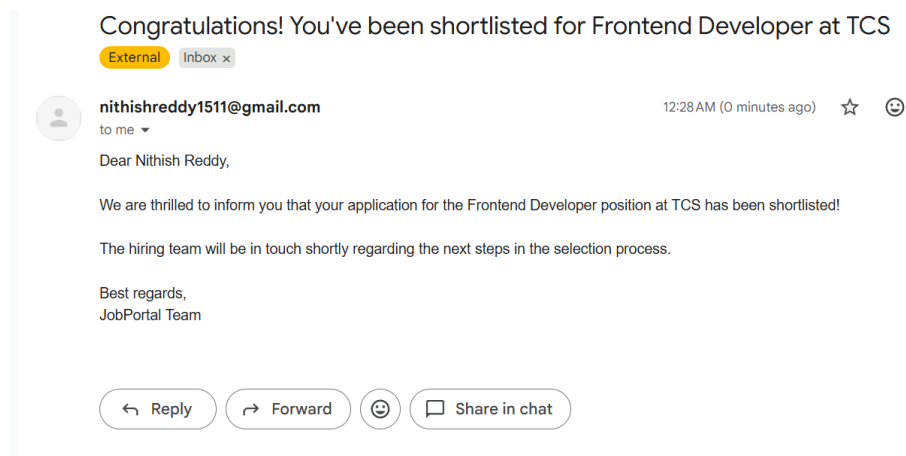


Figure 5.6: Shortlisting Email Notification

Figure 5.6 shows the email notification sent to users when they are shortlisted, ensuring timely communication.

Chapter 6

CONCLUSION AND FUTURE ENHANCEMENTS

6.1 Conclusion

The JobPortal application successfully demonstrates the power of the Spring Boot ecosystem in building enterprise-grade recruitment systems. By integrating secure authentication, automated email notifications, and efficient file storage, the project provides a comprehensive solution for modern hiring needs. The application is highly scalable and provides a solid foundation for any institutional recruitment drive.

6.2 Future Enhancements

- **Cloud Integration:** Migrate file storage to Amazon S3 for better reliability and scalability.
- **Resume Parsing:** Integrate AI/NLP tools to automatically extract

skills from student resumes.

- **Interview Scheduling:** Add a module for scheduling video interviews and sending calendar invites.
- **Advanced Analytics:** Provide admins with graphical insights into application trends and hiring rates.

6.3 SOURCE CODE IMPLEMENTATION

6.3.1 Backend Controller and Business Logic (UserController.java)

This section contains the core backend logic for the Job Portal, including the request mapping for user dashboards, job search functionality, and the logic for handling job applications with resume uploads using Spring Boot's multipart file handling.

```
1 package com.jobportal.controller;
2
3 import com.jobportal.entity.Job;
4 import com.jobportal.entity.JobApplication;
5 import com.jobportal.entity.User;
6 import com.jobportal.service.*;
7 import org.springframework.beans.factory.annotation.Autowired;
8 import org.springframework.security.core.Authentication;
9 import org.springframework.stereotype.Controller;
10 import org.springframework.ui.Model;
11 import org.springframework.web.bind.annotation.*;
12 import org.springframework.web.multipart.MultipartFile;
13
```

```

14 @Controller
15 @RequestMapping("/user")
16 public class UserController {
17
18     @Autowired
19     private JobService jobService;
20
21     @Autowired
22     private ApplicationService applicationService;
23
24     @Autowired
25     private UserService userService;
26
27     @Autowired
28     private FileStorageService fileStorageService;
29
30     @GetMapping("/dashboard")
31     public String dashboard(@RequestParam(required = false) String
        query,
32                             Model model, Authentication authentication
        ) {
33         model.addAttribute("jobs", jobService.searchJobs(query));
34
35         User student = userService.findByEmail(authentication.getName
            ()).orElse(null);
36         if (student != null) {
37             model.addAttribute("myApplications",
38                             applicationService.getApplicationsByStudent(student.
39                                     getId()));
40             return "user-dashboard";
41         }
42
43         @PostMapping("/apply/{jobId}")
44         public String applyForJob(@PathVariable Long jobId,

```

```

45         @ModelAttribute JobApplication
46             application,
47         @RequestParam("resumeFile")
48             MultipartFile resumeFile,
49         Authentication authentication) {
50
51     User student = userService.findByEmail(authentication.getName
52         ()) .orElse(null);
53     Job job = jobService.getJobById(jobId);
54
55     if (student != null && job != null) {
56         String fileName = fileStorageService.storeFile(resumeFile
57             );
58
59         application.setStudent(student);
60         application.setJob(job);
61         application.setResumePath(fileName);
62
63         applicationService.saveApplication(application);
64     }
65
66     return "redirect:/user/dashboard?applied=true";
67 }
68 }

```

Listing 6.1: UserController.java Implementation

6.3.2 Database Repository Layer (JobRepository.java)

The repository layer handles direct interaction with the MySQL database using Spring Data JPA. This interface allows for complex queries like searching for jobs by title or company name.

```
1 package com.jobportal.repository;
2
3 import com.jobportal.entity.Job;
4 import org.springframework.data.jpa.repository.JpaRepository;
5 import org.springframework.data.jpa.repository.Query;
6 import java.util.List;
7
8 public interface JobRepository extends JpaRepository<Job, Long> {
9
10     @Query("SELECT j FROM Job j WHERE j.title LIKE %?1% OR j.
11           companyName LIKE %?1%")
12     List<Job> searchJobs(String query);
13 }
```

Listing 6.2: JobRepository.java

Chapter 7

ADDRESSING COMPLEX ENGINEERING PROBLEM AND SDG

Level	Attribute	Characteristics	WKs	Justification
WP1	Depth of Knowledge	Requires in-depth engineering knowledge	WK3, WK4, WK5, WK6	Uses React.js, Node.js, Express, and MySQL for full-stack development
WP2	Conflicting Requirements	Technical and non-technical conflicts	WK4, WK5, WK6	Balances usability and system security
WP3	Depth of Analysis	Analytical and design thinking	WK3, WK4, WK5	Includes database design and API integration
WP4	Familiarity	Partially new problems	WK4, WK8	Real-time booking and email features
WP5	Applicable Codes	Limited standard solutions	WK6	Custom ticket booking logic
WP6	Stakeholder Needs	Multiple stakeholders involved	WK5, WK6, WK7	Admin and user interaction
WP7	Interdependence	System-level integration	WK3, WK5, WK6	Frontend, backend, and database integration

Table 7.1: Mapping of Project to Complex Engineering Problem (CEP)

7.1 Mapping of the Project to Complex Engineering Activities (CEA)

EA No.	Attribute	Characteristics	Justification
EA1	Range of Resources	Use of modern technologies	The system uses React.js, Node.js, Express, MySQL, and Nodemailer for full-stack development
EA2	Level of Interaction	Complex module interaction	Handles login, event management, booking, payment simulation, and email notifications
EA3	Innovation	Application of modern techniques	Implements real-time booking updates and automated email confirmation
EA4	Societal Impact	Benefits users	Simplifies event registration and improves accessibility for users
EA5	Familiarity	Beyond standard systems	Combines multiple modules like authentication, booking, and notification into one platform

Table 7.2: Mapping of Project to Complex Engineering Activities (CEA)

7.2 SDG Mapping (CEP Section)

SDG No.	SDG Title	Relevance to the Project
SDG 8	Decent Work and Economic Growth	Supports employment and skill development through event participation and management
SDG 9	Industry, Innovation and Infrastructure	Promotes digital transformation using modern web technologies and automation
SDG 10	Reduced Inequalities	Provides an accessible platform for all users regardless of location
SDG 11	Sustainable Communities	Encourages organized and efficient event management within institutions

Table 7.3: SDG Mapping for the Project

REFERENCES

- [1] LinkedIn Job Portal. A professional networking platform that allows users to create profiles, connect with professionals, and apply for jobs. Available at: <https://www.linkedin.com>
- [2] Naukri Job Portal. A leading Indian job portal that enables users to upload resumes, search for jobs, and receive recruiter notifications. Available at: <https://www.naukri.com>
- [3] Indeed Job Portal. A global job search engine that aggregates job listings from multiple sources and allows easy application. Available at: <https://www.indeed.com>
- [4] Glassdoor Job Portal. Provides job listings along with company reviews, salary insights, and interview experiences shared by employees. Available at: <https://www.glassdoor.com>
- [5] Monster Job Portal. An online employment platform offering job listings, resume services, and career advice tools. Available at: <https://www.monster.com>

- [6] Shine Job Portal. An Indian job portal that provides job listings, career guidance, and resume building services. Available at: <https://www.shine.com>
- [7] Spring Boot Documentation. Official documentation providing guidance for building production-ready applications using the Spring Boot framework. Available at: <https://spring.io/projects/spring-boot>
- [8] Spring Framework Documentation. Comprehensive documentation for developing Java applications using the Spring Framework. Available at: <https://spring.io/projects/spring-framework>
- [9] Thymeleaf Documentation. A modern server-side Java template engine used for building dynamic web applications. Available at: <https://www.thymeleaf.org>
- [10] MySQL Documentation. Official reference for MySQL database, covering installation, configuration, and query management. Available at: <https://dev.mysql.com/doc/>